Big Data Analytics - 22AIE312

Unit 1

Introduction

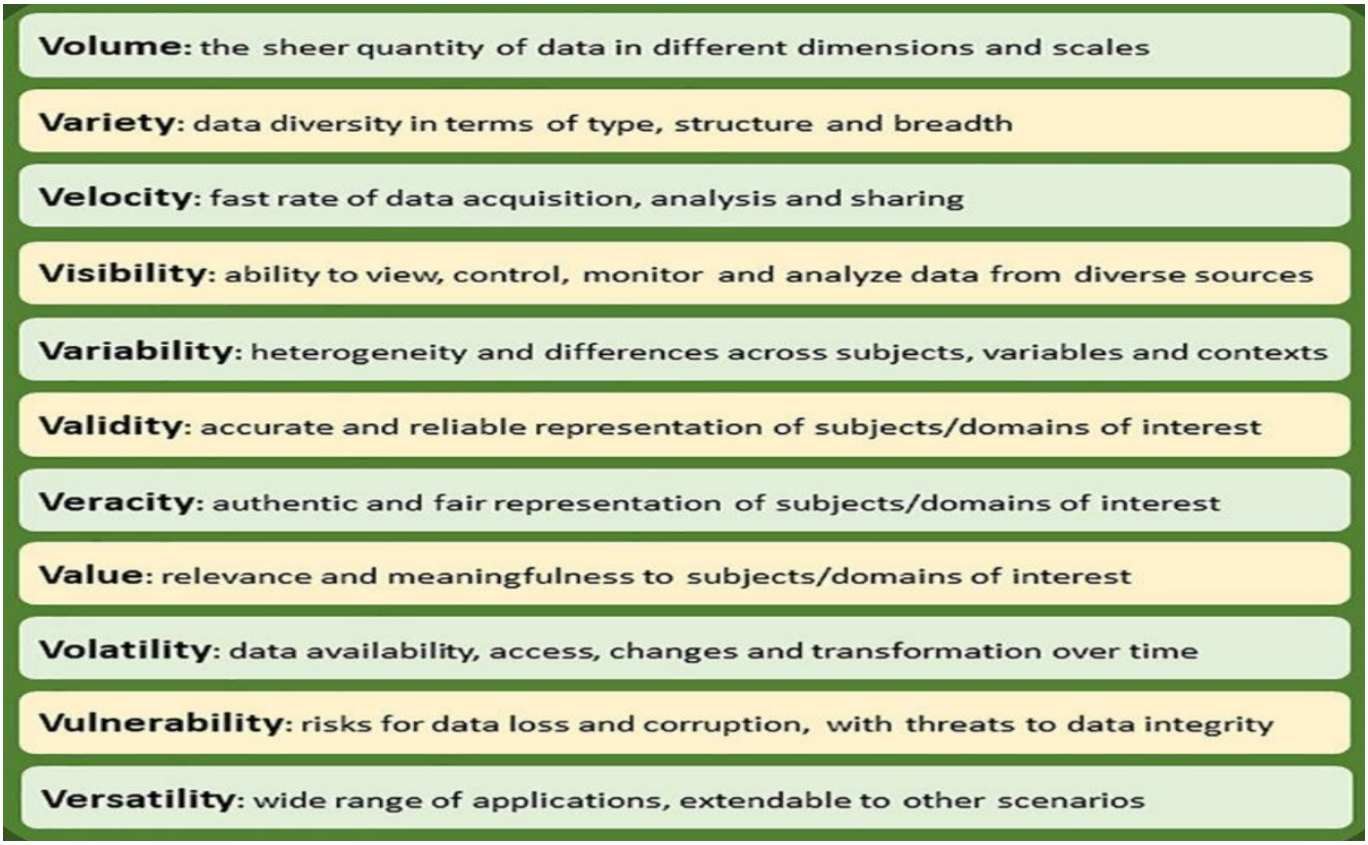
Big data is high volume, high velocity and high variety information assets that demand cost effective, innovative forms of information processing for enhanced insight and decision making.John Mashey coined the term "Big Data".

Three Characteristics of Big Data - 3 V's

Doug Laney introduced the 3 Vs

1. **Volume** - Data quantity
2. **Velocity** - Data speed
3. **Variety** - Data types

Various V's associated with Big Data

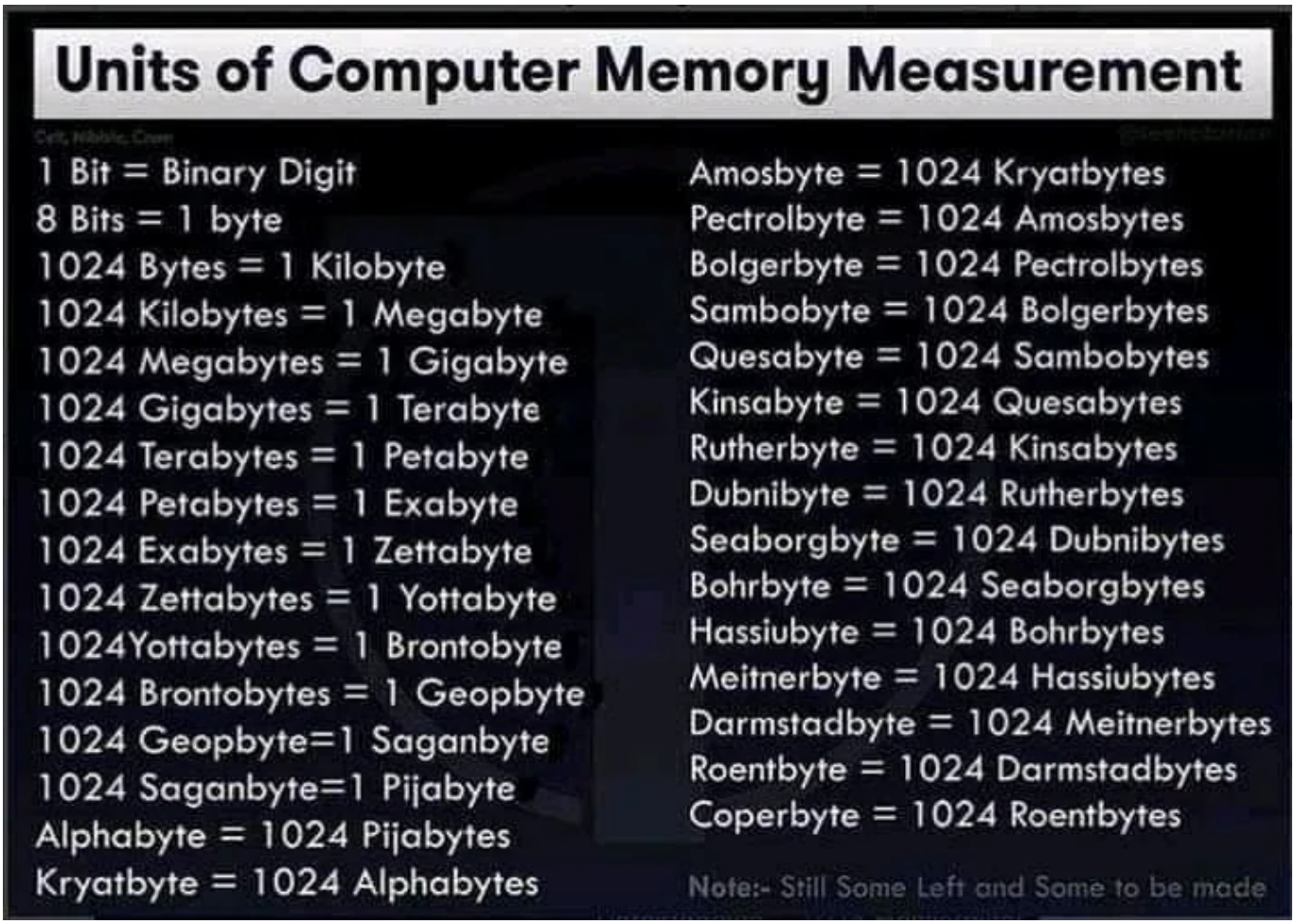


Types of Data

1. **Structured Data** - The data that resides in a fixed field within a record. Eg: spreadsheets, sql, oracle etc.
2. **Semi-Structured Data** - It is not bound by any rigid schema for data storage and handling. Data is stored in key value pairs. Eg: e-mail, XML, JSON etc.

3. **Unstructured Data** - It is the kind of data that doesn't adhere to any definite schema or set of rules. Its arrangement is unplanned. Eg: images, audio, video etc.

Typical Sizes



Unit	Bytes	Example
Bits	2	Machine code
Bytes	2^{3}	Single character
Kilobytes	2^{10} 10^{3}	Text document
Megabites	2^{20} 10^{6}	MP3 audio files
Gigabytes	2^{30} 10^{9}	Movies
Terabytes	2^{40} 10^{12}	Hard disk drive
Petabytes	2^{50} 10^{15}	Google services
Exabytes	2^{60} 10^{18}	Internet traffic
Zettabytes	2^{70}	Global digital data

Unit	Bytes	Example
Brontobytes	10^{27}	

Data vs Info

Data	Information
Data is a collection of facts	Information puts those facts into context
Raw and unorganized	Organized
Typically comes in the form of graphs, numbers, figures, or statistics	Typically presented through words, language, thoughts, and ideas
Data does not depend on information	Information depends on data
It is not sufficient for decision-making	You can make decisions based on information

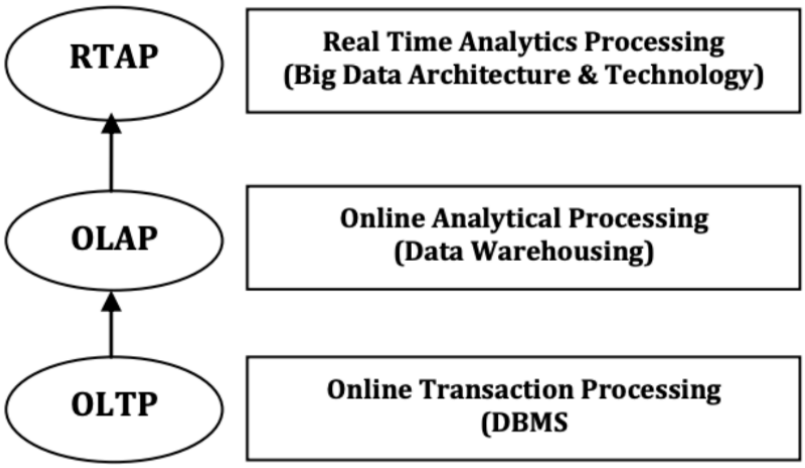
Data Analytics

1. **Descriptive Analytics** tells you what happened in the past.
2. **Diagnostic Analytics** helps you understand why something happened in the past.
3. **Predictive Analytics** predicts what is most likely to happen in the future.
4. **Prescriptive Analytics** recommends actions you can take to affect those outcomes.

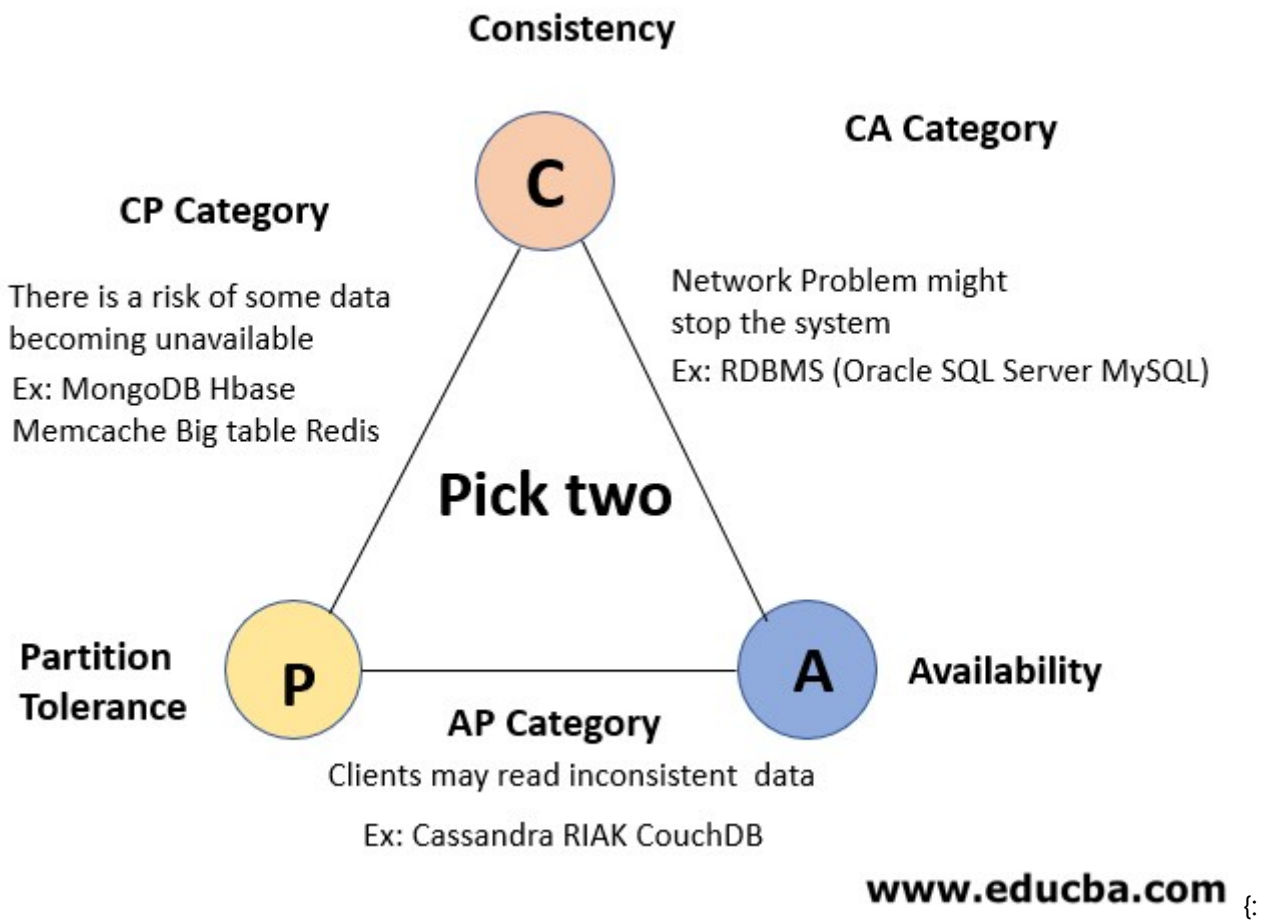
C's of data quality

- **3 Cs:** Completeness, Consistency, Cleanliness/Correctness
- **4 Cs:** Consistency, Conformity, Completeness, Currency
- **7 Cs:** Accuracy, Completeness, Consistency, Credibility, Compliance, Clarity, Timeliness/Currency

---



HTAP-Hybrid transactional /Analytical Processing



{: width="500" height="350"} *CAP Theorem*

Challenges in Big Data

1. **Scale:** vertically or horizontally to take care of massive storage requirement
2. **Security:** lack of proper Authentication and Authorization Mechanisms
3. **Partition Tolerant:** to take care of hardware and software failure
4. **Data Quality:** Maintain data accuracy, completeness, timeliness etc

BASE Model

- **Basically Available (B):** The system guarantees availability, but data might be stale.
- **Soft State (S):** System state can change over time, even without input, due to data updates and replication.
- **Eventually Consistent (E):** Data will become consistent across all nodes over time, but not immediately.
- Use Case: Ideal for social media or e-commerce where temporary inconsistencies are acceptable for higher availability and scalability

---

NoSQL

- Stands for **Not Only SQL**
- NoSQL is a non-relational database management system
- Instead of using structured tables to store multiple related attributes in a row, NoSQL databases use the concept of a key/value store

Types of NoSQL

Key value data store
- Riak
- Redis
- Membase

Document data store
- MongoDB
- CouchDB
- RavenDB

Column-oriented data store
- Cassandra
- HBase
- HyperTable

Graph based data store
- InfiniteGraph
- Neo4
- Allegro Graph

SQL vs NoSQL

Aspect	SQL (Relational)	NoSQL (Non-relational)
Data Structure	Tables with rows and columns	Document-based, key-value, column-family, or graph-based
Schema	Fixed schema (predefined structure)	Flexible schema (dynamic and adaptable)
Scalability	Vertically scalable (upgrading hardware)	Horizontally scalable (adding more servers)
Data Integrity	ACID-compliant (strong consistency)	BASE-compliant (more available, less consistent)
Query Language	SQL (Structured Query Language)	Varies (e.g., MongoDB uses its own query language)
Performance	Efficient for complex queries and transactions	Better for large-scale data and fast read/write operations
Use Case	Best for transactional systems (banking, ERP, etc.)	Ideal for big data, real-time web apps, and data lakes
Examples	MySQL, PostgreSQL, Oracle, MS SQL Server	MongoDB, Cassandra, CouchDB, Neo4j

NoSQL vs NewSQL

NoSQL	NewSQL
NoSQL is a schema-free database.	NewSQL is schema-fixed as well as a schema-free database.
It is horizontally scalable.	It is horizontally scalable.
It possesses automatically high-availability.	It possesses built-in high availability.
It supports cloud, on-disk, and cache storage.	It fully supports cloud, on-disk, and cache storage.
It promotes CAP properties.	It promotes ACID properties.
Online Transactional Processing is not supported.	Online Transactional Processing is fully supported.
There are low-security concerns.	There are moderate security concerns.
Use Cases: Big Data, Social Network Applications, and IOT.	Use Cases: E-Commerce, Telecom industry, and Gaming.
Examples : DynamoDB, MongoDB, RaveenDB etc.	Examples : VoltDB, CockroachDB, NuoDB etc.

Hadoop

Hadoop Core Components

1. **HDFS** (Data Storage Layer)

- Hadoop Distributed File System handles is the core component of Hadoop ecosystem which stores huge data.
- **NameNode (master):** Stores metadata (data about data) and requires fewer resources.
- **DataNodes (slaves):** Store actual data on commodity hardware, making Hadoop cost-effective.

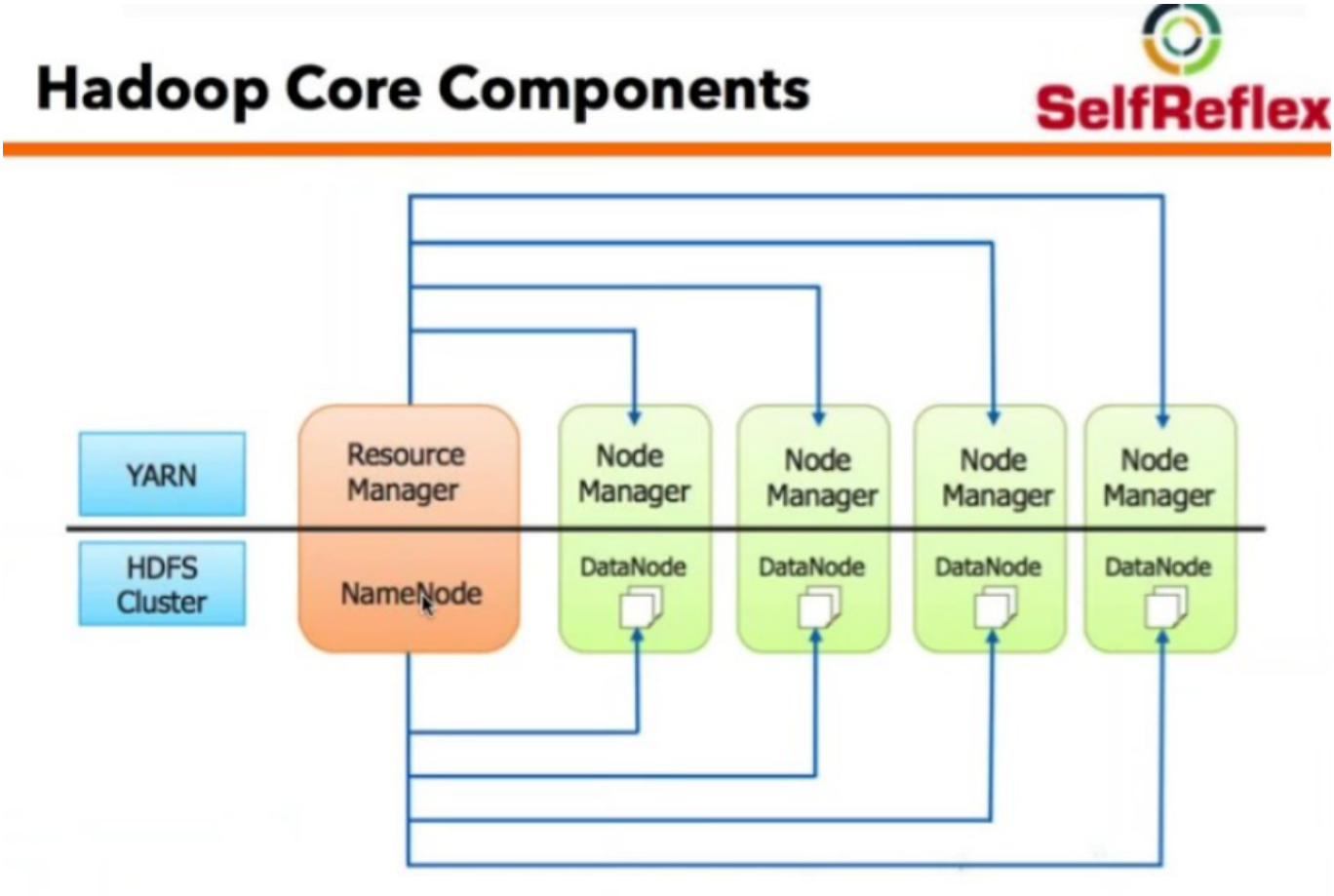
2. **YARN**

- Yet Another Resource Navigator is a resource management framework that allocates cluster resources to various applications.
- **ResourceManager(RM):** Allocates resources to various applications in the system.
- **NodeManager(NM):** Manages resources (CPU, memory, etc.) on individual nodes and reports to ResourceManager.
- **ApplicationMaster(AM):** Acts as a bridge between ResourceManager and NodeManager, handling resource negotiation for each application.
- **Container:** A fraction of node resources allocated by the RM toan application to run tasks

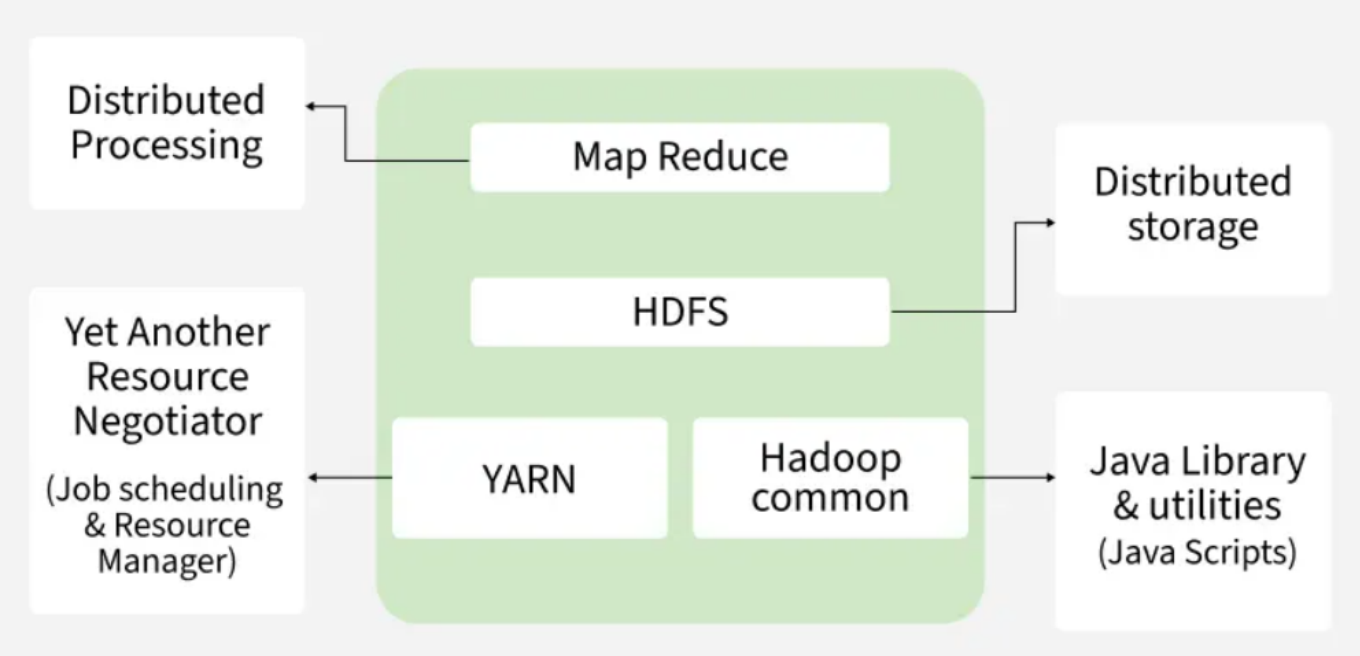
3. **MapReduce** (Data Processing Layer)

- Enables distributed and parallel data processing on large datasets.

- Any **Job Schedule** goes through MapReduce which belongs to Data Node
- **Map():** Processes input data into key value pairs.
- **Reduce():** Takes the output from Map(), aggregates the data and summarizes it into a smaller, consolidated set of results.

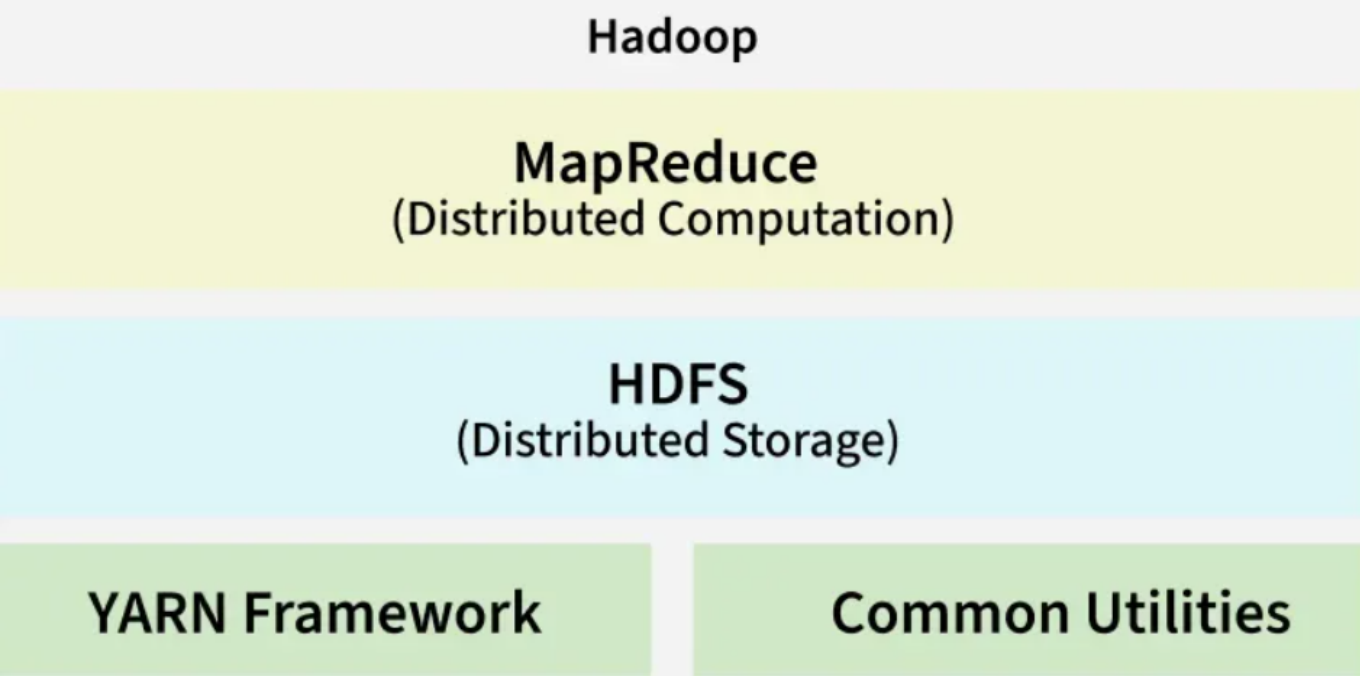


{: width="500" height="350"}



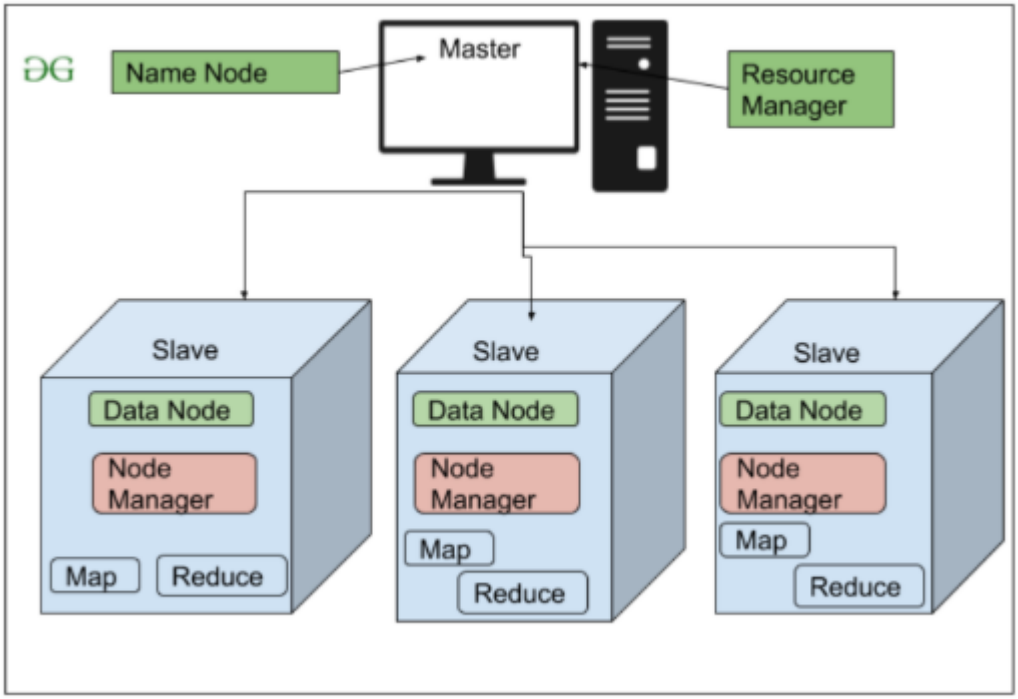
{: width="500" height="350"}

How does Hadoop work?



{: width="500" height="350"}

HDFS Daemons



{:height="500" width="500"}

- **Blocks** - Minimum amount of data that can be read or written
- **Name Node** - Master node of the system
- **Data Nodes** - Slaves which are deployed on each machine and provide the actual storage
- **Secondary Name Node** - It acts as a checkpoint in HDFS which helps NN but cannot replace it. It uploads new FS image to NN and Transaction Log is purged. It does not provide true backup if NN fails.
- **Standby Name Node** - A hot standby for High Availability(HA). It provides true live backup, synchronized and ready for immediate activation, crucial for HA.
- **Rack Awareness** - Choosing data node racks that are closest to each other which improves cluster performance by reducing network traffic

- **Replication** - Creating a copy of the data block which is also used for backup purposes
- **Replication Factor** - Number of times the data block was copied
- **Rack** - The rack is a physical collection of nodes in our Hadoop cluster (30 to 40)

Hadoop 2.0

- YARN is called Operationg System
- There is no Job Tracker and Task Tracker
- Hive is a relational database developed by FB
- Sqoop is a CLI used to import tables in Hadoop
- Spark: Provides fast, in-memory data processing.
- Hive & Pig: High-level tools for querying and analyzing large datasets.
- HBase: A NoSQL database for real-time read/write access.
- Mahout & Spark MLlib: Libraries for scalable machine learning.
- Solr & Lucene: Tools for full-text search and indexing.
- Zookeeper: Manages coordination and configuration across the cluster.
- Oozie: A workflow scheduler for managing Hadoop jobs.

Functions of NameNode

- **Manages File System Namespace** - Maps a file name to a set of blocks, Maps a block to the DataNodes where it resides
- **Cluster Configuration Management**
- **Replication Engine for Blocks**
- **To ensure high availability** - you need both an active NameNode and a standby NameNode

NameNode Metadata

- **Metadata in Memory** • The entire metadata is in main memory
- **Types of metadata** • List of files • List of Blocks for each file • List of DataNodes for each block • File attributes, e.g. creation time, replication factor
- **A Transaction Log** • Records file creations, file deletions etc

DataNode

- **Block Server** • Stores data in the local file system • Stores metadata of a block • Serves data and metadata to Clients
- **Block Report** • Periodically sends a report of all existing blocks to the NameNode
- **Facilitates Pipelining of Data** • Forwards data to other specified DataNodes

Heartbeats

• DataNodes send hearbeat to the NameNode periodically (Once every 3 seconds) • NameNode uses heartbeats to detectDataNode failure

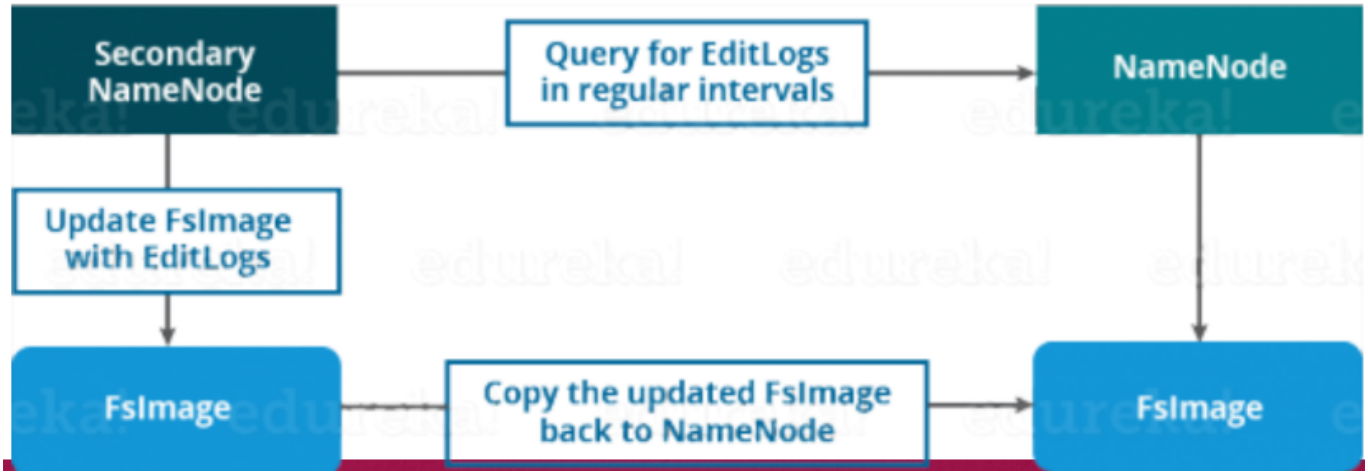
Replica Placement Strategy

- **First replica** is placed on the same node as the client.
- **Second replica** is placed on a node that is present on different rack.

- **Third replica** is placed on the same rack as second, but on a different node in the rack.
- This strategy provides good reliability, availability and fault tolerance
- The current default HDFS block placement policy guarantees that a block's 3 replicas will be placed on at least 2 racks.
- Specifically one replica is placed on one rack and the other two replicas are placed on another rack during write pipeline

Secondary NameNode

- **Role:** A helper process, not a backup.
- **Function:** Periodically fetches fsimage and editlogs from the NameNode, merges them, and saves the new fsimage, preventing editlogs from growing too large.
- **Failover:** Does not provide automatic failover; if the NameNode fails, the cluster stops until manual recovery.
- **Use Case:** Older Hadoop versions or non-HA setups, primarily for managing checkpointing



Standby NameNode

- **Role:** A hot standby for High Availability (HA).
- **Function:** Receives all metadata updates in real-time from the Active NameNode, keeping its own state synchronized and ready to take over.
- **Failover:** Provides fast, automatic failover (seconds) if the Active NameNode crashes, ensuring minimal disruption.
- **Use Case:** Modern HDFS clusters with High Availability enabled.

Key Distinction between two SNNs

Secondary NameNode	Standby NameNode
They are used in older / non-HA setups	They are essential for HA (High Availability) clusters
Acts as a helper to the Active NN to prevent the edit logs from becoming too large	It serves as a hot standby that mirrors the Active NameNode's memory state for immediate failover
It is not a true backup for failover	It is a true live backup

Map Reduce

Map task takes care of loading, parsing, transforming and filtering

Each Map-Reduce task is broken into the following phases:

1. **Record reader**

- Input – Line by line text from the input file.
- Output – Forms the key-value pairs.

2. **Mapper**

- The Map phase takes input from the Record Reader, processes it, and produces the output as another set of key value pairs.
- It reads each key-value pair, divides each word from the value using String Tokenizer

3. **Combiner / Semi Reducer**

- The main function of a Combiner is to summarize the map output records with the same key.
- The Combiner phase takes each key-value pair from the Map phase, processes it, and produces the output as key-value collection pairs.

4. **Partitioner / Reducer**

- The number of partitioners is equal to the number of reducers.
- A partitioner partitions the key-value pairs of intermediate Map-outputs. It partitions the data using a user-defined condition, which works like a hash function.
- The primary role of reducer is to reduce a set of intermediate values to a smaller set of values.
- The reducer has three primary phases: i. Shuffle and Sort ii. Reduce iii. Output format

5. **Record Write**

- Input – Each key-value pair from the Reducer phase along with the Output format.
- Output – It gives you the key-value pairs in text format.

Key Drawbacks of MapReduce

- **High Data Movement & Network Congestion:** The shuffle and sort phase moves large amounts of data across nodes, consuming bandwidth and degrading performance as cluster size increases.

- **Complex Programming Model:** Writing MapReduce jobs in Java is complex and lacks built-in support for high-level language analytics (SQL)

- **Lack of Caching:** MapReduce does not cache data, meaning the same dataset is re-read from the disk for every subsequent job.

- **High Latency & Batch Processing Only:** MapReduce is designed for massive batch jobs and cannot handle real-time or streaming data.

- **Performance Bottleneck (Disk I/O):** It heavily relies on reading writing data to disk during intermediate steps, which is far slower than in-memory alternatives like Apache Spark.